hexens x Layerswap

Security Review Report for Layerswap

March 2026

Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - updateWhitelistedAddress should revert if oldAddr and newAddr are identical
    - Use Ownable2Step instead of Ownable for better security
    - depositERC20 with fee-on-transfer tokens will emit an incorrect amount

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This audit reviews the LayerswapDepository, a smart contract that forwards native and ERC20 tokens to whitelisted Layerswap receiver addresses. It accepts deposits from users and forwards funds immediately to pre-approved (whitelisted) Layerswap solver addresses. The contract never holds funds, as all assets are forwarded atomically within the same transaction.

Our review was conducted over a one-day period and involved a comprehensive analysis of the contract.

During the assessment, we did not identify any high- or medium-severity vulnerabilities. We reported three informational findings.

All identified issues were remediated by the development team and subsequently verified by our auditors.

Following the remediation phase, we conclude that the protocol's overall security posture and code quality have improved as a result of this audit.

3. Security Review Details

Review Led by

Trung Dinh, Lead Security Researcher

Scope

The analyzed resources are located on:

🔗 https://github.com/layerswap/layerswap-depository/blob/e53e0b9b5145a60fbe45fe132b82740c5560f7b7/src/LayerswapDepository.sol

The issues described in this report were fixed in the following commit:

🔗 https://github.com/layerswap/layerswap-depository/commit/a7a4ccd89f0fb5046f8d0053283da6e36c6b638c

Changelog

23 March 2026	Audit start
24 March 2026	Initial report
24 March 2026	Revision received
26 March 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

- **Issue Symbolic Codes**

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	0
Low	0
Informational	3
Total:	**3**

Informational

Fixed

6. Weaknesses

This section contains the list of discovered weaknesses.

LYSWP3-1 | updateWhitelistedAddress should revert if oldAddr and newAddr are identical

Fixed

Severity:	Informational	Probability:	Rare	Impact:	Informational

Path:

src/LayerswapDepository.sol#L103

Description:

In the **updateWhitelistedAddress()** function, when **oldAddr == newAddr,** the function still succeeds: the address is removed from the set and then re-added, and **AddressUpdatedInWhitelist(addr, addr)** is emitted. Although the whitelist remains unchanged, the emitted event is misleading.

```
function updateWhitelistedAddress(address oldAddr, address newAddr) external onlyOwner {
    if (!_whitelist.remove(oldAddr)) revert NotWhitelisted();
    if (newAddr == address(0)) revert ZeroAddress();
    if (newAddr == address(this)) revert InvalidReceiver();
    if (!_whitelist.add(newAddr)) revert AlreadyWhitelisted();
    emit AddressUpdatedInWhitelist(oldAddr, newAddr);
}
```

Remediation:

Consider adding a check to revert when both addresses are the same:

```
function updateWhitelistedAddress(address oldAddr, address newAddr) external onlyOwner {
+   if (oldAddr == newAddr) revert InvalidReceiver();
    if (!_whitelist.remove(oldAddr)) revert NotWhitelisted();
    if (newAddr == address(0)) revert ZeroAddress();
    if (newAddr == address(this)) revert InvalidReceiver();
    if (!_whitelist.add(newAddr)) revert AlreadyWhitelisted();
    emit AddressUpdatedInWhitelist(oldAddr, newAddr);
}
```

LYSWP3-2 | Use Ownable2Step instead of Ownable for better security

Fixed

Severity:	Informational	Probability:	Rare	Impact:	Informational

Path:

src/LayerswapDepository.sol#L15

Description:

The **LayerswapDepository** contract inherits from OpenZeppelin's **Ownable** contract, which allows for immediate ownership transfers. This creates a risk where ownership could be accidentally transferred to an incorrect or inaccessible address, resulting in the contract becoming permanently locked without an owner.

```
contract LayerswapDepository is Ownable, Pausable, ReentrancyGuard {
```

Remediation:

Consider replacing the **Ownable** inheritance with **Ownable2Step**:

```
- import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
+ import { Ownable2Step } from "@openzeppelin/contracts/access/Ownable2Step.sol";

- contract LayerswapDepository is Ownable, Pausable, ReentrancyGuard {
+ contract LayerswapDepository is Ownable2Step, Pausable, ReentrancyGuard {
```

This change maintains the same functionality while adding an additional security layer to ownership transfers. The new owner will need to call **acceptOwnership()** to complete the transfer process, preventing accidental transfers to incorrect addresses.

LYSWP3-3 | depositERC20 with fee-on-transfer tokens will emit an incorrect amount

Fixed

Severity:	Informational	Probability:	Rare	Impact:	Informational

Path:

src/LayerswapDepository.sol#L80-L82

Description:

In the **depositERC20()** function, if the **token** is a fee-on-transfer token, the **receiver** will receive less than **amount**, but the event logs the full **amount**. The off-chain system would have an incorrect amount of the actual transfer.

```
/// @notice Forwards ERC20 tokens from caller to a whitelisted receiver.
///         Caller must approve this contract before calling.
/// @param id Unique identifier for this deposit (correlates with off-chain order)
/// @param token ERC20 token address
/// @param receiver Whitelisted address to receive the funds
/// @param amount Amount of tokens to forward
function depositERC20(bytes32 id, address token, address receiver, uint256 amount)
    external
    nonReentrant
    whenNotPaused
{
    if (token == address(0)) revert ZeroAddress();
    if (!_whitelist.contains(receiver)) revert NotWhitelisted();
    if (amount == 0) revert ZeroAmount();

    // Emit before external call (CEI pattern)
    emit Deposited(id, token, receiver, amount);

    IERC20(token).safeTransferFrom(msg.sender, receiver, amount);
}
```

Remediation:

Consider measuring the balance delta and emit the actual received amount.

hexens x Layerswap